

Tokens

- Small piece/unit of text that LLM or AI model processes as an input or produces as an output.
- That unit can be
 - part of word(sub-word) e.g. ing, able, ful etc.
 - complete word e.g. cat, dog, the, play
 - whitespace
 - number
 - special character, emoji etc.

❖ Tokenization:

- Process of converting raw text into tokens that the model can process.
- LLMs are mathematical models so it only understands numbers.
- To solve this problem, we use Tokenizer which acts as language translator that breaks text into pieces and assigns Token ID to each piece from massive, pre-built master dictionary i.e. Vocabulary/Vocab dictionary.
- **Example:** Imagine models vocabulary looks like –

Token (piece of text)	Token ID
trans	412
form	105
er	89
s	21
ing	34

If we input word “transformers”, the tokenizer matches the biggest fragment it can find: [“trans”, “form”, “er”, “s”] (4 tokens) and if we input “transforming” then: [“trans”, “form”, “ing”] (3 tokens).

- If a model has vocabulary size of 100,000 it means model only knows 100,000 specific token variations. These can be whole words, sub-words, punctuations, spaces combined with text, numbers etc. Everything we ever type into AI model must be forced into combination of those 100,000 tokens.
- **Example:** Vocabulary size of different models
 - Older/English-focused models (LLaMA2, GPT-3.5): ~32,000 to 50,000 tokens
 - Modern standard models (LLaMA3): ~128,000 tokens

- Large Multilingual models (OpenAI o200k_base, LLaMA4): ~200,000+ tokens
- Modern LLMs use sub-word tokenization, which breaks a text into common patterns, prefixes, suffixes, and punctuation. This will help in balancing vocabulary size and handle rare or out-of-vocabulary words.
- **Example:** A sub-word tokenizer might break playing into ["play", "ing"] (2 tokens), while "hello" might remain ["hello"] (1 token). This result may vary, depending on the type of tokenizer and vocabulary.
- Why LLM don't use words i.e. Word-based tokenization?
Consider words like play, playing, played, player, playground. If every variation becomes own token, then size of vocabulary becomes unnecessarily large. Also, word-based tokenizer would trouble in handling words it has never seen like SolidGoldMagicarp, the model may glitch out, panics, or start speaking non-sense. So, instead of requiring every word to exist in the vocabulary, the tokenizer can construct word from smaller known pieces i.e. ["Solid", "Gold", "Magic", "arp"].

❖ Token ID:

- Unique number i.e. numerical identifier that LLM model assign to a specific token in tokenizer's vocabulary so it can process languages mathematically.
- As LLM is a mathematical model so it only understands numbers. So, after tokenizer generates Token IDs and then this IDs are mapped to numerical vector representations called "Embeddings".
- **Example:**
Suppose we enter "AI is changing the world!", then tokenizer may produce something like ["AI", " is", " changing", " the", " world", " !"] (6 tokens – In modern LLMs a whitespace before word is merged into next token itself).
Then each token is assigned with unique Token ID i.e. [17527, 382, 13784, 290, 5992, 0] (This IDs and overall token count may vary, it depends on the tokenizer and model we are using).

❖ Generic Rules of Token:

- Rule of thumb: For English text, 1 token is about 4 characters i.e. 100 tokens are nearly equal to 75 words. These are rough estimates for English text, not fixed rules.
- Common vs Rare words: Common words like the, and, cat, or the words which we use in day-to-day life usually count as 1 token. Rare, long, or

complex words may break into multiple sub-words (It depends on tokenization method model is using), like “unbelievable” which is a long word might be split down into [“un”, “believe”, “able”] (3 tokens).

- **Punctuation Count:** Punctuation mark like (? “ ” ; ,) can be represented as its own individual token and has unique Token ID or as a part of token, depending on the tokenizer.
- **Space Count:** In the modern tokenizers like Open AI's cl100k_base, a space before word in a sentence is merged into token itself.
 - The word “hello” is one token
 - The word “hello” is one token, but both have completely different Token ID.
- **Case Sensitivity Matters:** Capitalization changes the Token ID. “apple” and “Apple” can have different Token IDs.

❖ Token Limits and Why Token Count Matters:

- **Context window:** This is the total number of tokens the model can process and use within a single request i.e. within single conversation (including our prompt or input and model's generated response). If conversation exceeds this limit, the model may need to remove or exclude some earlier content from the context supplied to model. Every model has a strict context window size ranges from 8000 to 128,000 tokens (It again depends on or varies by model, so always checks the documentation for the specific model we are using).
- **Cost:** AI companies like Open AI, Anthropic, Google don't bill us by the word or by the hour. They usually bill us per 1 Million tokens. Also, input and output token may have different prices.
- **Language Bias:** Most models are trained mostly on English, non-English languages like Hindi, Spanish etc. required more number of tokens.

Example: The “hello” can be 1 token but its Hindi translation “नमस्ते” can have 4 to 6 tokens. This makes models in non-English languages slower and more expensive.

❖ Practical Visualization:

Step1: Open either [OpenAI tokenizer](#) or [TikTokenizer](#)(for this we can select a particular model we want test our text/prompt in) in browser.

Step2: Enter prompt/sentence like “The capital of India is Delhi”. Observe total number of characters, number of tokens, how the sentence is divided into individual tokens, and Token IDs.

Step3: Test different types of text like normal words, with punctuation, long words, numbers, code, different languages, emojis etc. Also, you can take one sentence and gradually modify it and observe how token count changes.

Step4: Notice the actual word count and token count.